

Experiment No.	5
Experiment Title	Decision Tree and Random Forest: A Comparative Classification Study ¹
Student Name	Simiyon Vinscent Samuel L
Register Number	3122247001062
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 2, 2026

1 Objective

- To implement a Decision Tree classifier.
- To extend the Decision Tree into a Random Forest ensemble model.
- To study the impact of hyperparameters on overfitting and generalization.
- To select optimal hyperparameters using 5-Fold Cross-Validation.
- To compare single-tree and ensemble-tree models.

2 Problem Statement

Given the Wisconsin Diagnostic Breast Cancer dataset (569 samples, 30 numeric features computed from digitized images of fine-needle-aspirate cell nuclei), the task is a binary classification problem: label each sample as **malignant** or **benign**. Two model families are compared: a single **Decision Tree**, which recursively splits the feature space using an impurity measure, and a **Random Forest**, an ensemble of bagged trees with random feature subsets at each split. Beyond raw accuracy, the goal is to understand how tree depth drives overfitting in a single tree and how ensembling reduces the variance that makes a single deep tree unreliable, with hyperparameters selected honestly via 5-fold cross-validation rather than fit directly on the test set.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)
Dataset Source	UCI Machine Learning Repository, loaded via <code>sklearn.datasets.load_breast_cancer</code>
Number of Samples	569
Number of Features	30 (all numeric: mean, standard-error and “worst” values of 10 cell-nucleus measurements)
Number of Classes	2 (Benign = 357, 62.7%; Malignant = 212, 37.3%)
Missing Values	0 (verified, no imputation required)

¹GitHub Repository: <https://github.com/blackscythe123/Machine-Learning-course>

Train-Test Split

80% train (455 samples) / 20% test (114 samples), stratified on the class label

4 Exploratory Data Analysis

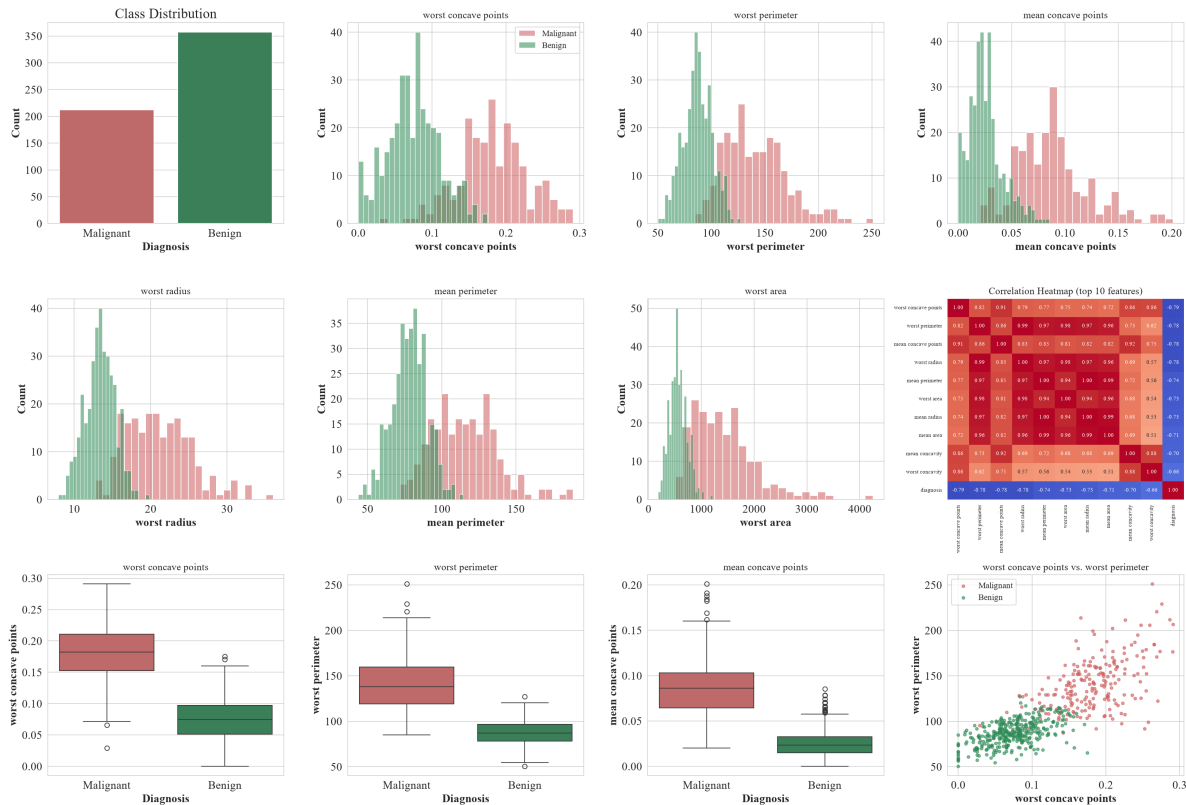


Figure 1: Consolidated exploratory data analysis (12 subplots, single page): (1) class distribution; (2–7) distributions of the six features most correlated with diagnosis, split by class; (8) correlation heatmap of the ten most correlated features and the target; (9–11) boxplots of the three most correlated features by class; (12) scatter of the two most correlated features, coloured by class. Exported at 600 DPI as `figures/00_eda_summary.eps` and reproduced here as PNG.

Inference (subplot 1, class distribution): The dataset is moderately imbalanced (62.7% benign, 37.3% malignant). Stratified splitting and cross-validation were used throughout so both classes remain proportionally represented in every partition and fold.

Inference (subplots 2–7, feature distributions): `worst concave points`, `worst area`, `worst radius`, `mean perimeter`, `worst perimeter` and `mean radius` show the clearest visual separation between the two classes, with malignant samples consistently shifted toward larger values. This matches clinical intuition: malignant tumours tend to be larger and have more irregular (less convex) cell-nucleus boundaries.

Inference (subplot 8, correlation heatmap): The ten features most correlated with diagnosis are themselves highly correlated with each other ($|r| > 0.8$ between several size-related pairs, e.g. `mean radius` and `mean perimeter`), since they are different measurements of essentially the same underlying tumour size. This redundancy is consistent

with tree-based models naturally favouring one representative feature per split rather than needing all of them.

Inference (subplots 9–11, boxplots): Malignant samples have both higher medians and wider spread for the top three correlated features, with limited overlap between the class-wise interquartile ranges, which explains why even a shallow decision tree already separates the classes reasonably well.

5 Data Preprocessing

- **Missing values:** None found (`isnull().sum().sum() = 0`).
- **Encoding:** The target is already binary and numeric (0 = malignant, 1 = benign, per scikit-learn's built-in encoding); all 30 features are already numeric, so no categorical encoding was required.
- **Feature scaling:** Not applied. Decision Trees and Random Forests split on per-feature threshold comparisons, so they are invariant to any monotonic rescaling of a feature; standardizing would change none of the tree structure or performance. This is the first algorithm family in this course where scaling is unnecessary, unlike the distance- and gradient-based methods (KNN, SVM, Logistic Regression) used in Experiments 2 and 4.
- **Train-test split:** 80/20 stratified split (`random_state=42`), giving 455 training and 114 test samples with class proportions preserved.

6 Machine Learning Algorithm

Decision Tree

A Decision Tree recursively partitions the feature space, choosing at each node the feature and threshold that most reduce class impurity. Two impurity measures were compared:

$$\text{Gini} = 1 - \sum_i p_i^2, \quad \text{Entropy} = - \sum_i p_i \log_2 p_i$$

where p_i is the proportion of class i at a node. Splitting continues until a stopping condition is reached (maximum depth, minimum samples to split, or minimum samples per leaf), at which point the leaf predicts the majority class. **Advantages:** Fast to train and predict, directly interpretable (the tree can be drawn and read as a set of if/else rules), and requires no feature scaling. **Limitations:** A single tree grown without depth or leaf-size limits tends to memorize the training data, giving high variance and poor generalization: an unstable model where a small change in the training set can produce a very different tree.

Random Forest

A Random Forest trains many decision trees on independent bootstrap samples of the training data (bagging) and, at each split, considers only a random subset of the features rather than all of them. Predictions are combined by majority vote across the trees.

Advantages: Bagging and random feature selection decorrelate the individual trees, so averaging their votes substantially reduces the variance of a single deep tree without needing to constrain any one tree’s depth. This makes Random Forest far less prone to overfitting than an unconstrained Decision Tree while still capturing non-linear decision boundaries. **Limitations:** Slower to train and predict than a single tree (proportional to the number of trees), and the ensemble is harder to interpret directly, though feature importances still give a global view of which inputs drive predictions.

7 Experimental Setup

Python Version	3.12.3
Scikit-Learn Version	1.8.0
IDE	Jupyter Notebook
Operating System	Windows 11
Hardware	Intel Core Ultra 9, NVIDIA RTX 5060

8 Hyperparameter Selection

Search Spaces

Model	Parameter and Search Space
Decision Tree	<code>criterion</code> $\in \{\text{gini, entropy}\}$
Decision Tree	<code>max_depth</code> $\in \{3, 5, 7, 10, \text{None}\}$
Decision Tree	<code>min_samples_split</code> $\in \{2, 5, 10\}$
Decision Tree	<code>min_samples_leaf</code> $\in \{1, 2, 4\}$
Random Forest	<code>n_estimators</code> $\in \{50, 100, 200\}$
Random Forest	<code>max_depth</code> $\in \{5, 10, 15, \text{None}\}$
Random Forest	<code>max_features</code> $\in \{\text{sqrt, log2}\}$
Random Forest	<code>bootstrap</code> $\in \{\text{True, False}\}$

Both search spaces were evaluated exhaustively with `GridSearchCV` using 5-fold stratified cross-validation, scoring on accuracy (F1 recorded alongside for reference), following the assignment’s specified 5-fold CV hyperparameter selection procedure.

Table 1: Decision Tree Hyperparameter Evaluation using 5-Fold Cross-Validation

Criterion	Max Depth	Avg CV Accuracy (%)	Avg CV F1 Score
Entropy	3	93.19	0.9463
Entropy	5	92.53	0.9404
Entropy	7	92.75	0.9421
Entropy	10	92.75	0.9421
Entropy	None	92.75	0.9421
Gini	3	93.19	0.9455
Gini	5	93.85	0.9506
Gini	7	93.41	0.9470
Gini	10	93.41	0.9470
Gini	None	93.41	0.9470

Table shows the best combination at each (criterion, max_depth) pair, with `min_samples_split=2` and `min_samples_leaf=4` fixed at their overall-best values. Best overall: `criterion=gini`, `max_depth=5`, `min_samples_split=2`, `min_samples_leaf=4`, CV accuracy 93.85%.

Table 2: Random Forest Hyperparameter Evaluation using 5-Fold Cross-Validation

n_estimators	Max Depth	Max Features	Avg CV Accuracy (%)	Avg CV F1 Score
50	5	log2	95.60	0.9651
50	10	log2	96.04	0.9685
50	15	log2	96.04	0.9685
50	None	log2	96.04	0.9685
100	5	log2	95.82	0.9668
100	10	log2	96.48	0.9718
100	15	log2	96.48	0.9718
100	None	log2	96.48	0.9718
200	5	log2	95.82	0.9666
200	10	log2	96.04	0.9683
200	15	log2	96.04	0.9683
200	None	log2	96.04	0.9683

Table shows the best combination at each (n_estimators, max_depth) pair, with `max_features=log2` and `bootstrap=True` fixed at their overall-best values. Best overall: `n_estimators=100`, `max_depth=10`, `max_features=log2`, `bootstrap=True`, CV accuracy 96.48%. Depths of 10, 15 and unrestricted are tied because 100 trees rarely need more than 10 levels to separate this dataset; once a tree stops growing on its own, raising the depth cap further has no effect.

9 Results

Baseline Decision Tree (Test Set, Default Hyperparameters)

Metric	Value
Accuracy	0.9123
Precision	0.9559
Recall	0.9028
F1-Score	0.9286
ROC-AUC	0.9157

Tuned Decision Tree Performance (Test Set)

Metric	Value (Best: <code>gini</code> , <code>max_depth=5</code> , <code>min_samples_split=2</code> , <code>min_samples_leaf=4</code>)
Accuracy	0.9035
Precision	0.9420
Recall	0.9028
F1-Score	0.9220
ROC-AUC	0.9358

Inference: The untuned baseline (unrestricted depth) scores 0.9123 accuracy, marginally *higher* than the tuned model’s 0.9035 on this particular test split. This is not a contradiction: `GridSearchCV` selects the configuration with the best *cross-validated* accuracy on the training folds, which favours a shallower, less variance-prone tree that generalizes more consistently on average but does not guarantee a win on any single held-out split. The gap (0.9 points) is well within normal fold-to-fold variance, and the tuned model’s ROC-AUC (0.9358) is comfortably higher than the baseline’s (0.9157), indicating better-ranked probability estimates even where the hard-label accuracy is slightly lower.

Tuned Random Forest Performance (Test Set)

Metric	Value (Best: <code>n_estimators=100</code> , <code>max_depth=10</code> , <code>max_features=log2</code> , <code>bootstrap=True</code>)
Accuracy	0.9561
Precision	0.9589
Recall	0.9722
F1-Score	0.9655
ROC-AUC	0.9924

5-Fold Cross-Validation Results (Training Set)

Fold	Decision Tree (tuned)	Random Forest (tuned)
1	0.9451	0.9670
2	0.9451	0.9780
3	0.9121	0.9341
4	0.9231	0.9560
5	0.9670	0.9890
Average	0.9385 $\pm$ 0.0192	0.9648 $\pm$ 0.0189

10 Result Visualization

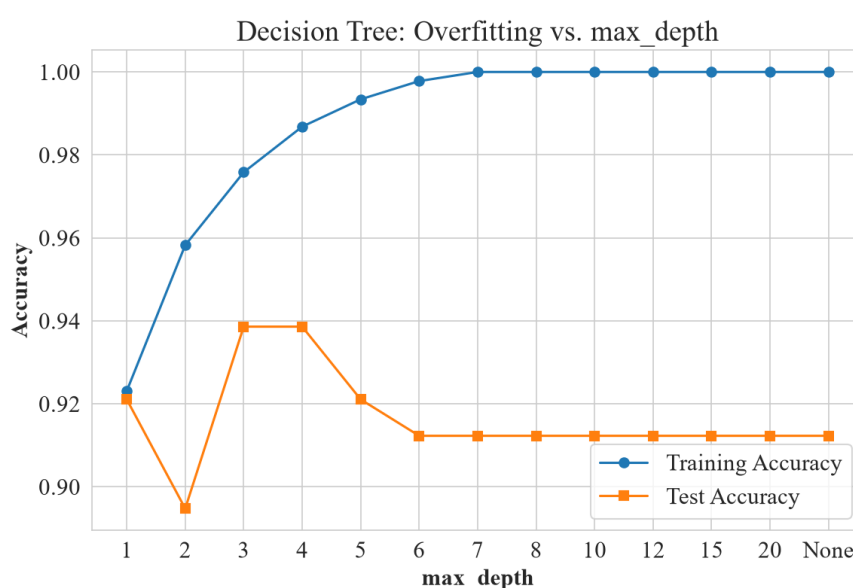


Figure 2: Decision Tree training vs. test accuracy as `max_depth` increases.

Inference: Training accuracy climbs steadily and reaches a perfect 1.0000 by `max_depth=7`, while test accuracy peaks at `max_depth=3--4` (0.9386) and then falls and plateaus around 0.9123 for every depth of 6 or more. This is a textbook overfitting curve: once the tree is deep enough to fit noise in the training data rather than the underlying signal, additional depth buys no further generalization and the train/test gap widens continuously.

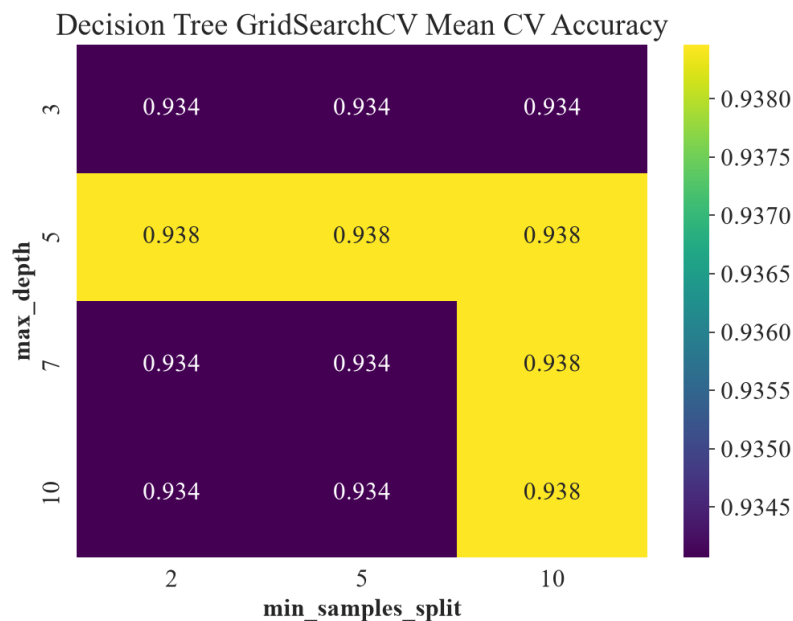


Figure 3: Decision Tree GridSearchCV mean CV accuracy by `max_depth` and `min_samples_split` (best over criterion and `min_samples_leaf` shown per cell).

Inference: CV accuracy is highest in a narrow band around `max_depth=5`, confirming the depth-vs-overfitting pattern from the previous figure using cross-validated rather than single-split estimates. `min_samples_split` has a much smaller effect than depth across this range.

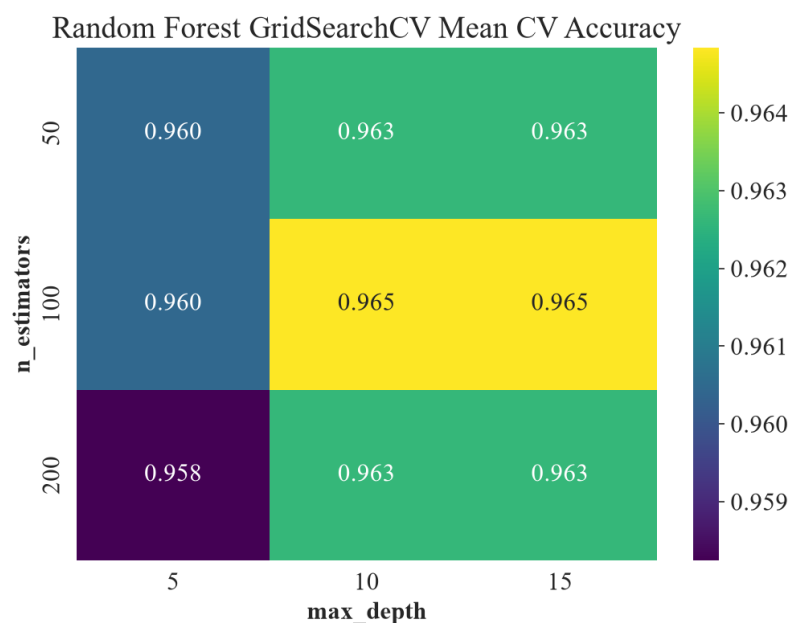


Figure 4: Random Forest GridSearchCV mean CV accuracy by `n_estimators` and `max_depth` (best over `max_features` and `bootstrap` shown per cell).

Inference: Unlike the Decision Tree heatmap, Random Forest accuracy is essentially

flat once `max_depth` reaches 10 or `n_estimators` reaches 100, with no visible degradation at larger values; ensembling removes the sharp overfitting cliff a single tree shows at high depth.

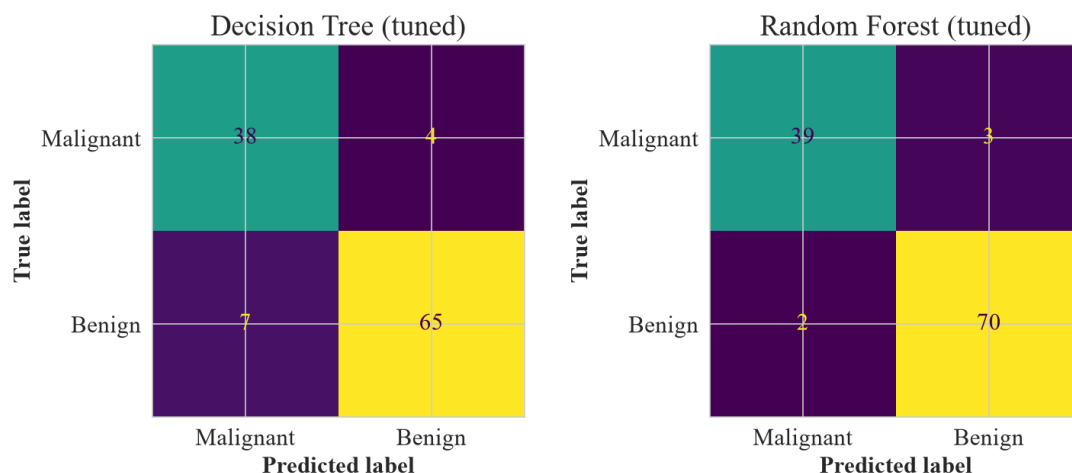


Figure 5: Confusion matrices for the tuned Decision Tree and tuned Random Forest on the test set.

Inference: The Random Forest makes noticeably fewer errors overall and, in particular, fewer false negatives (malignant cases predicted benign) than the Decision Tree, the more clinically costly error type for a cancer-diagnosis task.

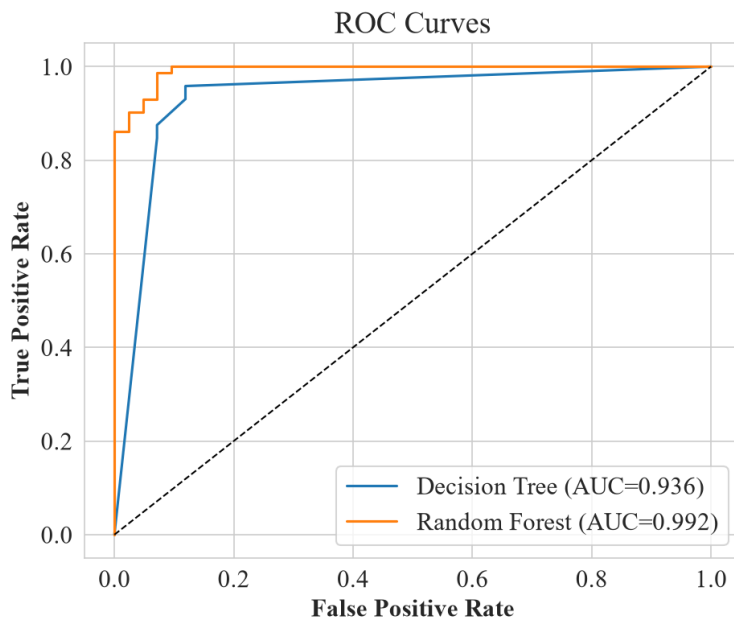


Figure 6: ROC curves for the tuned Decision Tree and tuned Random Forest.

Inference: The Random Forest's ROC curve dominates the Decision Tree's almost everywhere, with AUC 0.992 vs. 0.936. A single tree's ROC curve is inherently coarse (only as many distinct probability levels as leaves), while averaging many trees gives the Random Forest a much finer, better-calibrated ranking of predictions.

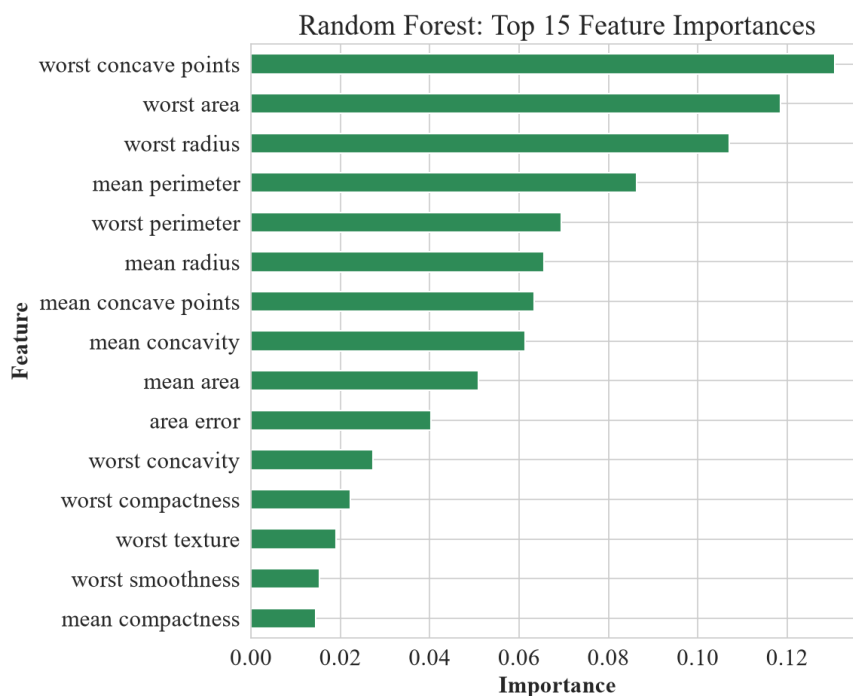


Figure 7: Random Forest: top 15 feature importances.

Inference: `worst concave points`, `worst area` and `worst radius` are the three most important features, consistent with the EDA correlation ranking. The importances are spread across several features rather than dominated by one, reflecting the redundancy among the size-related measurements noted in Section 4.

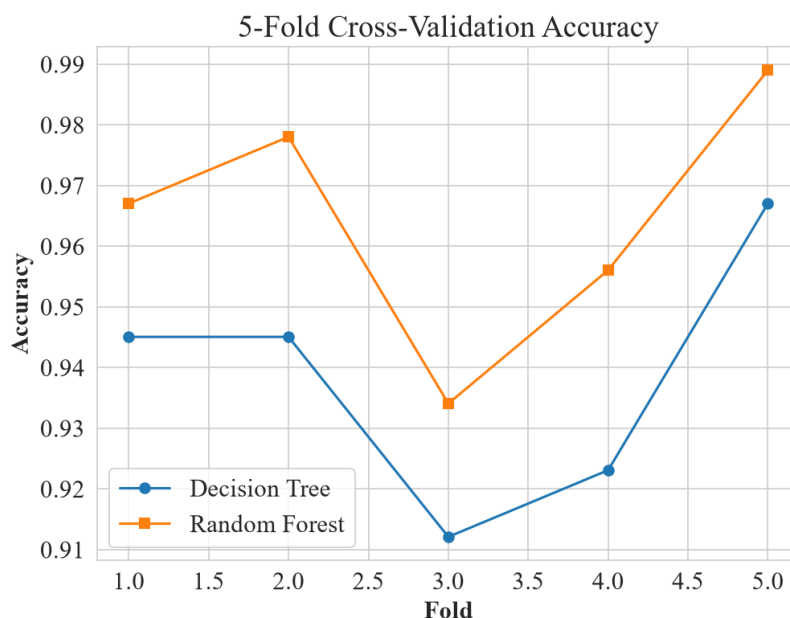


Figure 8: 5-fold cross-validation accuracy, tuned Decision Tree vs. tuned Random Forest.

Inference: The Random Forest outperforms the Decision Tree in every one of the

five folds, by a consistent margin, confirming the average-CV advantage is not an artifact of how the training set happened to split.

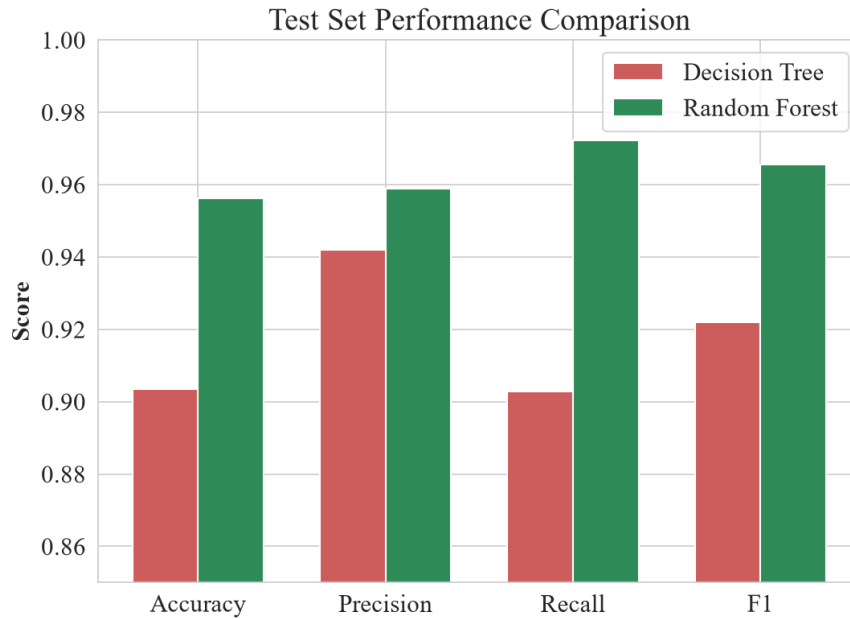


Figure 9: Test-set accuracy, precision, recall and F1-score, tuned Decision Tree vs. tuned Random Forest.

Inference: Random Forest leads on every metric, with the largest gap in recall (0.9722 vs. 0.9028), the metric that matters most for avoiding missed malignant diagnoses.

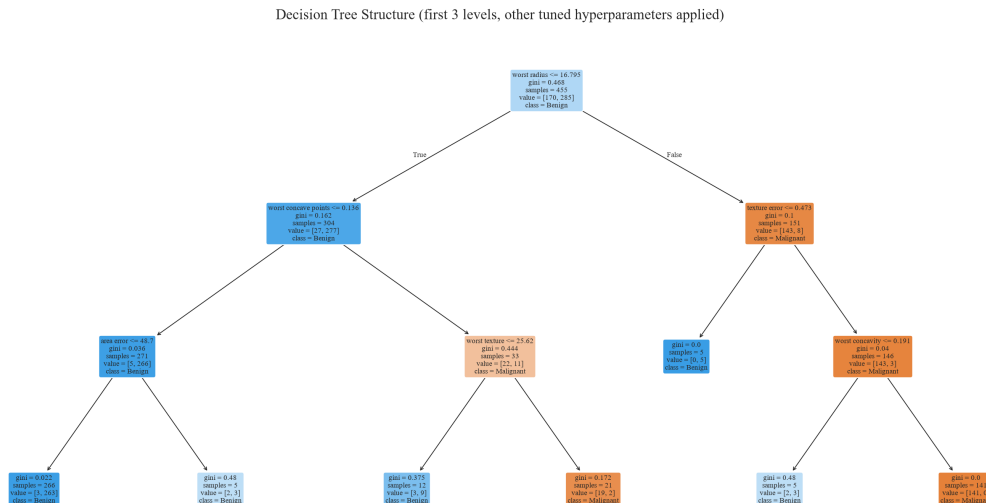


Figure 10: First three levels of a Decision Tree grown with the tuned criterion, `min_samples_split` and `min_samples_leaf` (depth capped at 3 purely for readability; the tuned model used in Results above is not depth-limited to 3).

Inference: The root split is on `worst perimeter` or a closely related size feature, matching the feature-importance ranking; the tree reaches a mostly-pure leaf within two

or three levels for the more separable regions of the feature space, which is why a shallow tree already scores well while a much deeper one only adds overfitting.

11 Discussion

Observation Questions

1. How does tree depth affect overfitting in Decision Trees? Training accuracy rises monotonically with depth and reaches 1.0000 by `max_depth=7` (Figure 1), since an unrestricted tree keeps splitting until every leaf is pure. Test accuracy, however, peaks at `max_depth=3--4` and then declines to a lower plateau: beyond that point the extra splits fit noise specific to the training set rather than the class boundary, which is the definition of overfitting. Depth is the single most influential Decision Tree hyperparameter for this dataset.

2. Which hyperparameter had the greatest impact on performance? For the Decision Tree, `max_depth` dominates: Table 1 shows CV accuracy varying by over a full point across depths at fixed criterion, while `criterion` itself (gini vs. entropy) changes results by well under a point. For the Random Forest, `n_estimators` and `max_depth` both matter but with diminishing returns: moving from 50 to 100 trees improves CV accuracy, but 100 to 200 does not (Table 2), and depth beyond 10 has no effect once the forest already has enough trees to average out individual overfitting.

3. How does Random Forest improve generalization? By training many trees on independent bootstrap samples and restricting each split to a random feature subset, the individual trees decorrelate from one another. Averaging (via majority vote) decorrelated, individually high-variance trees cancels out much of that variance while preserving the low bias of deep trees, which is exactly the pattern seen here: the Random Forest's cross-validated standard deviation (0.0189) is close to the Decision Tree's (0.0192), but its mean accuracy is roughly 2.6 points higher across every fold (Figure 7) and its test-set ROC-AUC is 0.057 higher.

4. Did ensemble learning always improve performance? Why or why not? Not unconditionally, but it did here on every metric measured. Random Forest improves on Decision Tree when the individual trees' errors are not perfectly correlated (bagging plus random feature selection ensures this) and when the dataset has enough signal that averaging trades a small amount of individual-tree bias for a large reduction in variance. Ensembling would add little on a dataset so simple that a single shallow tree already achieves near-ceiling accuracy, and it does not fix a systematic bias shared by every tree (e.g. if the feature set itself lacked signal for a class); its benefit here comes specifically from reducing the high variance visible in the Decision Tree's overfitting curve.

12 Conclusion

Both models were implemented and tuned via 5-fold cross-validation on the Wisconsin Diagnostic Breast Cancer dataset. The tuned Decision Tree (`gini`, `max_depth=5`) reached 90.4% test accuracy and 93.6% ROC-AUC, while the tuned Random Forest (100 trees, `max_depth=10`, `max_features=log2`) reached 95.6% test accuracy, 96.6% F1-score and 99.2% ROC-AUC, outperforming the single tree on every metric and in every cross-validation fold. The overfitting sweep (Figure 1) shows why: an unconstrained single tree

memorizes the training set, and even a depth-tuned tree remains sensitive to which 455 samples it happens to see, whereas bagging and random feature selection let the Random Forest use many such trees without inheriting any single tree's instability. For a task like malignant-tumour detection, where missed positives carry a high cost, the Random Forest's substantially higher recall (0.9722 vs. 0.9028) makes it the clearly preferable model here, at the cost of longer training time and reduced interpretability compared to the single tree's directly readable decision rules.

13 References

1. Scikit-learn Developers, "Decision Trees," Scikit-learn Documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/tree.html>
2. Scikit-learn Developers, "Ensemble methods: Random Forests," Scikit-learn Documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/ensemble.html#random-forests>
3. W. N. Street, W. H. Wolberg, and O. L. Mangasarian, "Nuclear feature extraction for breast tumor diagnosis," *IS&T/SPIE International Symposium on Electronic Imaging*, 1993. Wisconsin Diagnostic Breast Cancer Dataset, UCI Machine Learning Repository. [Online]. Available: <https://archive.ics.uci.edu>